



OOP

(Object oriented programming)

By Alejandro Martin





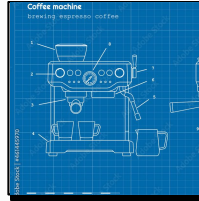
1.- Basic concepts

01



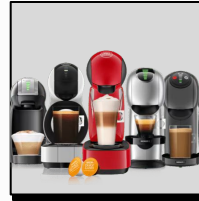
Objects

02



Classes

03



Instances

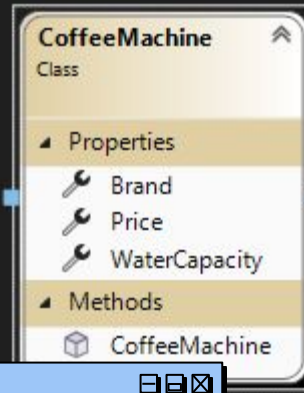
04



Constructs

Back

Next



```
3 references
public string Brand { get; set; }
3 references
public int WaterCapacity { get; set; }
3 references
public double Price { get; set; }
2 references
public CoffeeMachine(string brand, int
{
    Brand = brand;
    WaterCapacity = waterCapacity;
    Price = price;
}
```

1.- Examples of basic concepts

```
Philips, 80, 99,99
Nespresso, 60, 69,99
```

```
CoffeeMachine MyCoffeeMachine = new CoffeeMachine("Nespresso", 60, 69.99);
CoffeeMachine YourCoffeeMachine = new CoffeeMachine("Philips", 80, 99.99);
```

Back

Next

2.- Encapsulation

- Public

- Private

```
public CoffeeMachine(string brand, int waterCapacity, double price)
{
    Brand = brand;
    WaterCapacity = waterCapacity;
    Price = price;
    WaterLevel = 0;
}
```

```
CoffeeMachine MyCoffeeMachine = new CoffeeMachine("Nespresso", 60, 69.99);
CoffeeMachine YourCoffeeMachine = new CoffeeMachine("Philips", 80, 99.99);
```

```
3 references
public string Brand { get; set; }
4 references
public int WaterCapacity { get; set; }
3 references
public double Price { get; set; }
```

CoffeeMachine
Class

- Fields
 - WaterLevel
- Properties
 - Brand
 - Price
 - WaterCapacity
- Methods
 - CoffeeMachine
 - MakeCoffee
 - RefillWater

```
private int WaterLevel;
```

```
public CoffeeMachine(string brand, int waterCapacity, double price)
{
    Brand = brand;
    WaterCapacity = waterCapacity;
    Price = price;
    WaterLevel = 0;
}
```

```
public void MakeCoffee()
{
    const string Made = "Here you go, one cup of coffee!";
    const string NoWater = "There's not enough water!";
    if (WaterLevel >= 1){
        Console.WriteLine(Made);
        WaterLevel--;
    }
    else
    {
        Console.WriteLine(NoWater);
    }
}

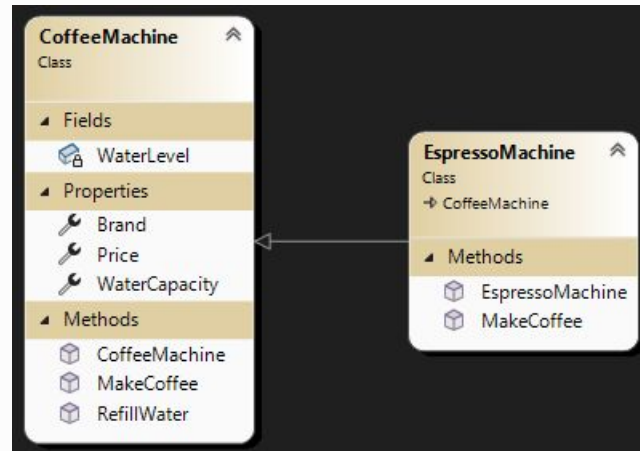
0 references
public void RefillWater()
{
    const string Refilled = "Water refilled!";
    WaterLevel = WaterCapacity;
    Console.WriteLine(Refilled);
}
```

Back

Next

3.- Inheritance and polymorphism

```
1 reference
public class EspressoMachine : CoffeeMachine
{
    0 references
    public EspressoMachine(string brand, int waterCapacity, double price) : base(brand, waterCapacity, price) {}
    1 reference
    public override void MakeCoffee()
    {
        Console.WriteLine("Here's your Espresso!!");
    }
}
```



Back

Next

4.- Helper Class

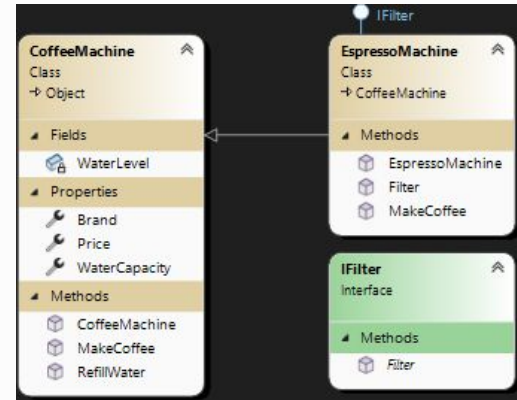
- Interfaces

```
1 reference  
public class EspressoMachine : CoffeeMachine, IFilter
```

- Common methods

```
public void Filter()  
{  
    Console.WriteLine("Putting filter into coffee machine");  
}
```

```
1 reference  
public interface IFilter  
{  
    1 reference  
    void Filter();  
}
```



5.- Types of relations



Association



Inheritance



Aggregation



Composition

Back

Next



Thanks for watching!

Have a nice day!!!



Back

Next